SMART ULTRASONIC SENSOR HOME SURVEILLANCE RADAR SYSTEM

Project Report

Heimdall's Eye — IoT Surveillance System

Course	ENG85 2130 — Internet of Things for Engineering Application
Project Title	Smart Ultrasonic Sensor Home Surveillance Radar System
Team Name	Team Rocket
Members	B6741037 — Kyaw Khant Lwin B6741068 — Paing Nyein Oo
Date	1st June 2026

1. Executive Summary

This report documents the design, development, and implementation of the Smart Ultrasonic Sensor Home Surveillance Radar System — internally codenamed Heimdall's Eye — developed by Team Rocket for the course ENG85 2130: Internet of Things for Engineering Application.

The system is an intelligent, low-cost IoT surveillance solution that uses an ESP32 microcontroller, HC-SR04 ultrasonic sensor, and SG90 servo motor to perform continuous 180° radar sweeps, detect intruding objects in real time, and deliver instant threat-level notifications to the user via Telegram. A live radar dashboard is rendered through Node-RED and is accessible remotely via mobile browser.

Key achievements of this project include: successful dual-mode (Patrol/Tracking) servo sweep control, three-level threat classification with Telegram alerts, a custom HTML5 Canvas radar visualization dashboard, bidirectional remote control through Telegram commands (/on, /off), and full system validation through physical distance testing.

2. Introduction and Motivation

2.1 Problem Statement

Security and privacy are growing concerns in modern daily life, particularly when individuals are away from their personal spaces. Conventional home security systems are often expensive, complex to install, or dependent on proprietary cloud services with monthly subscription fees. There is a clear demand for affordable, responsive, and remotely accessible surveillance systems that can be built and maintained by technically literate users.

2.2 Project Concept

Inspired by Heimdall — the all-seeing Norse guardian — the team designed a smart radar surveillance system that mimics the scanning behaviour of a radar station: a rotating sensor continuously sweeps the environment, detects any object that crosses a defined perimeter, and immediately alerts the owner. The system requires no cloud subscription, runs on local Wi-Fi and MQTT, and can be controlled entirely from a smartphone.

2.3 Why Ultrasonic over PIR

The team initially considered using both the HC-SR501 PIR sensor and the HC-SR04 ultrasonic sensor. However, the PIR sensor proved physically incompatible with servo motor mounting due to its wide-angle dome and large PCB form factor. The HC-SR04, with its compact and directional beam, was selected as the sole detection sensor because it can be rigidly mounted on the servo horn and provides precise distance measurements at every swept angle.

3. Objectives and Goals

3.1 Primary Objectives

- Develop a smart ultrasonic radar surveillance system using ESP32 and IoT technologies.
- Integrate HC-SR04 ultrasonic sensor, SG90 servo motor, Node-RED, MQTT, and Telegram.
- Perform real-time object detection and distance measurement across a 180° sweep.
- Provide live radar visualization accessible on both desktop and mobile browsers.
- Deliver automated threat-level warning notifications to the user's mobile phone.

3.2 System Goals

- Achieve continuous 180° environmental scanning via servo-mounted HC-SR04.
- Detect objects within a 0–400 cm effective range.
- Enforce a virtual security perimeter at 200 cm.
- Automatically switch between Patrol Mode (slow scan) and Tracking Mode (high-speed scan) based on object proximity.
- Implement Smart Semi-Tracking to narrow the scan window (±20°) around detected objects for precision monitoring.
- Classify and deliver three-level threat notifications (Suspicious, Approaching, Territory Breach) via Telegram.
- Support bidirectional remote control: /on and /off commands via Telegram chatbot.
- Enable real-time mobile monitoring through Google Chrome on any device connected to the same Wi-Fi network.

4. System Components

The system is composed of six core hardware and software components, each fulfilling a distinct role in the overall IoT pipeline:

Component	Type	Role
ESP32 Microcontroller	Hardware	Main processing unit: runs C++ firmware, controls servo, reads HC-SR04, publishes MQTT data via Wi-Fi
HC-SR04 Ultrasonic Sensor	Hardware	Measures distance using ultrasonic pulses (Trig/Echo); mounted on servo horn for rotating coverage
SG90 Servo Motor	Hardware	Rotates 0°–180° to sweep the sensor across the full surveillance arc
Node-RED Dashboard	Software	Subscribes to MQTT data, processes threat levels, renders live radar animation, controls Telegram integration

Component	Type	Role
Telegram Chatbot (RadarBot)	Software/IoT	Sends threat alerts to user's phone; receives /on and /off remote control commands
Mobile Browser (Google Chrome)	Interface	Allows remote access to the Node-RED dashboard over Wi-Fi from any smartphone or tablet

5. Hardware Architecture and Circuit Design

5.1 Component Selection and Rationale

The ESP32 was selected as the core microcontroller for its built-in dual-core processor, native Wi-Fi and Bluetooth capability, and broad compatibility with the Arduino IDE and C++ ecosystem. It provides sufficient GPIO pins for both the HC-SR04 (Trig + Echo) and the servo PWM signal, while simultaneously maintaining a Wi-Fi connection for MQTT communication.

The HC-SR04 operates at 5V logic level, while the ESP32 GPIO pins are 3.3V tolerant. To prevent damage to the ESP32's Echo input pin, a resistor voltage divider was implemented using a 1kΩ and 2kΩ resistor pair, reducing the 5V Echo signal to approximately 3.3V before it enters the ESP32.

5.2 Pin Assignment Table

Signal	ESP32 Pin	Connected To	Note
Trig (Trigger)	GPIO 5	HC-SR04 Trig Pin	Output: 10µs HIGH pulse to initiate ultrasonic burst
Echo (Receive)	GPIO 18	HC-SR04 Echo Pin (via divider)	Input: receives reflected pulse; 3.3V logic via 1kΩ/2kΩ divider
Servo PWM	GPIO 13	SG90 Signal Wire	PWM output at 50 Hz; 1ms–2ms pulse width for 0°–180°
VCC (Sensor)	5V Rail	HC-SR04 VCC	HC-SR04 requires 5V supply; breadboard power rail
VCC (Servo)	5V Rail	SG90 Red Wire	Servo powered from 5V; yellow signal wire goes to GPIO 13
GND	GND	All components	Common ground for HC-SR04, SG90, and ESP32

5.3 Voltage Divider Calculation

The Echo pin of the HC-SR04 outputs a 5V pulse. The ESP32 GPIO maximum input is 3.3V. A resistor divider was used to step down the voltage:

- R1 = 1kΩ (connected between HC-SR04 Echo output and ESP32 GPIO 18)
- R2 = 2kΩ (connected between ESP32 GPIO 18 and GND)
- Output voltage = 5V × (2000 / (1000 + 2000)) = 5 × 0.667 = 3.33V ≈ 3.3V

This safely brings the Echo signal within the 3.3V tolerance of the ESP32 GPIO inputs.

6. ESP32 Firmware Design (C++)

6.1 Firmware Overview

The ESP32 firmware was developed in C++ using the Arduino IDE framework. The firmware handles four core tasks concurrently: servo sweep and motor control, ultrasonic distance measurement, operating mode management, and MQTT data publication over Wi-Fi.

6.2 Distance Measurement

The HC-SR04 is triggered by a 10-microsecond HIGH pulse on the Trig pin. The sensor emits an ultrasonic burst at 40 kHz and returns a HIGH pulse on the Echo pin whose duration corresponds to the time of flight for the sound wave. Distance is calculated using the standard formula:

```
Distance (cm) = duration (µs) × 0.0343 / 2
```

The formula uses the speed of sound (343 m/s = 0.0343 cm/µs at room temperature) and divides by 2 to account for the round trip (outgoing + return). A pulseIn() timeout of 30,000 µs was implemented to prevent the firmware from hanging when no echo is received.

6.3 Servo Sweep and Scanning Modes

- Patrol Mode (> 200 cm): The servo sweeps smoothly from 0° to 180° and back at a slow, consistent speed. This mode is active by default when no objects are detected within the 200 cm security boundary.
- Tracking Mode (≤ 200 cm): When an object crosses the 200 cm threshold, the firmware switches to high-speed rotation, improving detection responsiveness and tracking accuracy.
- Smart Semi-Tracking: When an object is actively detected, the servo narrows its sweep window to ±20° around the last known detection angle, providing tighter, more focused monitoring of the intruding object.

6.4 MQTT Data Publication

At each sweep step, the firmware packages the following data into a JSON payload and publishes it to the MQTT broker (HiveMQ public cloud broker) on the topic radar/data:

```
{ "angle": 90, "distance": 85.3, "mode": "TRACKING", "threat": 2 }
```

The ESP32 also subscribes to the topic radar/cmd to receive remote /on and /off control commands from the Node-RED Telegram integration. On receiving {cmd: “off”}, the servo stops and data publication halts.

6.5 Threat Level Classification

Threat Level	Name	Distance Range	Action
Level 0	Clear	> 200 cm	No alert; Patrol Mode continues
Level 1	Suspicious	100–200 cm	Telegram warning: object detected with angle and distance
Level 2	Approaching	50–100 cm	Telegram alert: object approaching with angle and distance
Level 3	Territory Breach	< 50 cm	Critical Telegram alert; red radar flash; phone vibration trigger

7. Node-RED Dashboard and IoT Integration

7.1 System Workflow Overview

The end-to-end system follows a publish-subscribe IoT architecture with the following pipeline:

- ESP32 Servo Sweep: The servo rotates 0°–180°; HC-SR04 measures distance at each angle step.
- MQTT Publish: ESP32 packages angle, distance, mode, and threat level as JSON and publishes to radar/data via HiveMQ.
- Node-RED Threat Router: A function node receives the JSON payload, evaluates threat level, and routes outputs to five destinations: radar canvas, distance gauge, status panel, Telegram alert, and alert log.
- Live Dashboard: Radar canvas animation, distance gauge, mode indicator, and alert status update in real time.
- Telegram Alerts: If threat level > 0, Node-RED sends a formatted alert message via the RadarBot Telegram bot.
- Remote Control: User sends /on or /off via Telegram. Node-RED publishes the command to radar/cmd. The ESP32 starts or stops scanning accordingly.

7.2 Node-RED Flow Description

The Node-RED flow consists of the following node groups:

MQTT Input and Data Processing

- MQTT In node subscribes to radar/data on HiveMQ broker.
- JSON Parse node converts the raw string payload into a JavaScript object.
- Threat Router function node evaluates distance, assigns threat level, and generates multiple output messages.

Live Radar Dashboard

- ui_template node renders the HTML5 Canvas radar animation: circular grid (100/200/300/400 cm rings), angle labels every 30°, an animated green sweep arm synchronized with the servo, and a red detection dot when an object is found.
- ui_gauge node displays the current measured distance in centimetres.
- ui_text nodes display mode (PATROL/TRACKING), threat level, and last alert status.

Telegram Integration

- Telegram Receiver node: Listens for incoming Telegram messages from the user.
- Command Parser function node: Identifies /on or /off, builds the MQTT payload {cmd: on/off}, and prepares a confirmation reply.
- MQTT Out node: Publishes the command to radar/cmd topic.
- Telegram Sender node: Sends the confirmation message and all threat-level alerts to the user's phone.
- 10-second alert cooldown: Prevents alert flooding when an object remains in the threat zone.

7.3 Remote Control Sequence

/ON Command Flow: User types /on → Telegram Server forwards to RadarBot → Node-RED Receiver catches message → Command Parser strips prefix and builds {cmd: on} → MQTT Out publishes to radar/cmd → HiveMQ forwards to ESP32 → ESP32 activates servo sweep and data publishing → Telegram confirms: "Radar turned ON — command sent to ESP32".

/OFF Command Flow: Same pipeline with {cmd: off}. ESP32 stops servo and data publishing. Dashboard freezes. Telegram confirms: "Radar turned OFF — command sent to ESP32". No further alerts are sent until the system is turned on again.

8. Team Contributions

Member	Role	Technical Contributions
Kyaw Khant Lwin (B6741037)	Hardware Architect & Embedded Software Engineer	Designed full circuit schematic and component selection (HC-SR04, SG90, ESP32). Physically wired all components on breadboard. Implemented 1kΩ/2kΩ voltage divider for 5V→3.3V Echo signal conversion. Developed complete C++ firmware: distance measurement, servo sweep, PATROL/TRACKING mode switching, Smart Semi-Tracking logic, threat classification, and MQTT JSON publishing over Wi-Fi.
Paing Nyein Oo (B6741068)	Software Engineer & IoT Dashboard Developer	Configured full Node-RED flow: MQTT subscription, JSON parsing, Threat Router function node, and routing to 5 output destinations. Developed HTML5 Canvas radar animation (sweep arm, detection dots, range rings, angle labels) inside ui_template node. Set up Telegram chatbot integration (RadarBot) with three-level threat notifications, 10-second cooldown mechanism, and bidirectional /on /off remote control.

9. AI Tools Disclosure

In accordance with the course's AI Disclosure Policy, the team used Claude Sonnet (Anthropic) as an AI coding and documentation assistant. The following table summarises the application and verification of AI-generated outputs:

Application Area	How AI Was Used	Verification Method
Firmware Architecture	Generated ESP32 C++ template for servo sweep logic, MQTT publish function, and PATROL/TRACKING mode switching algorithm.	Cross-checked distance formula (duration × 0.0343 / 2) against HC-SR04 datasheet. Physically validated with ruler at 10 cm, 50 cm, and 100 cm.
Coding Assistance	Assisted in debugging the ESP32 pulseIn() timeout issue that caused firmware freezing on no-echo events.	Serial monitor output in Arduino IDE confirmed correct behaviour during live sensor testing.
Dashboard Code	Generated the Node-RED ui_template HTML5 Canvas radar animation code (sweep	Visually verified animation synchronisation with physical servo movement during live system operation.

Application Area	How AI Was Used	Verification Method
	arm, range rings, detection dots).	
Documentation	Assisted in structuring system architecture documentation, Node-RED flowsheet descriptions, and wiring table annotations.	All technical facts reviewed and confirmed against actual hardware and software configurations by team members.

10. System Validation and Testing

10.1 Hardware Validation

- Distance Measurement Accuracy: Known distances of 10 cm, 50 cm, and 100 cm were measured using a ruler. The sensor output was compared to the expected value. Results confirmed the formula (duration × 0.0343 / 2) to be accurate within ±1.5 cm across all three test distances.
- Voltage Divider Verification: The Echo pin voltage was measured with a multimeter. Without the divider, the signal measured 4.95V. With the 1kΩ/2kΩ divider, the signal was measured at 3.28V, safely within the 3.3V tolerance of the ESP32 GPIO.

10.2 Firmware Validation

- Servo Sweep: The servo was confirmed to sweep continuously from 0° to 180° and back in Patrol Mode with no skipping or stuttering.
- Mode Switching: Placing an object at 180 cm (inside the 200 cm boundary) confirmed the firmware correctly switched from PATROL to TRACKING mode. Serial monitor output verified the mode change.
- MQTT Publishing: MQTT Explorer confirmed correct JSON payloads being received at radar/data with accurate angle, distance, mode, and threat level values.

10.3 Dashboard Validation

- Radar Animation: The sweep arm on the Node-RED dashboard was visually confirmed to synchronise with the physical servo position.
- Detection Dot: Placing an object at 85 cm triggered a red detection dot to appear at the correct angle and distance on the radar canvas.
- Mobile Access: The Node-RED dashboard was successfully accessed on a smartphone via Google Chrome over the local Wi-Fi network.

10.4 Telegram Integration Validation

- Level 1 Alert: Object placed at 150 cm. Telegram received: “Unknown object detected at 150 cm and angle X°”.
- Level 2 Alert: Object placed at 75 cm. Telegram received: “Object is approaching at 75 cm and angle X°”.
- Level 3 Alert: Object placed at 17.5 cm. Telegram received: “Unknown object has entered your territory at 17.5 cm and angle X°”. Red radar flash confirmed on dashboard.
- /on and /off Commands: Commands sent via Telegram were confirmed to start and stop the radar sweep with correct confirmation replies.

11. Results and Discussion

11.1 Achieved Outcomes

The final system successfully achieved all primary objectives and goals set at the outset of the project. The Heimdall’s Eye radar system functions as a fully operational IoT surveillance platform with the following confirmed capabilities:

- Continuous 180° environmental scanning via servo-mounted HC-SR04.
- Real-time distance mapping published to MQTT at every sweep step.
- Automatic mode switching between Patrol and Tracking based on object proximity.
- Smart Semi-Tracking to narrow scan window around a detected object.
- Three-level threat classification with automated Telegram notifications.
- Live HTML5 Canvas radar dashboard accessible on mobile and desktop browsers.
- Bidirectional remote control via Telegram /on and /off commands.
- Full system validated through physical testing at three threat distance thresholds.

11.2 Challenges Encountered

- 5V/3.3V Logic Mismatch: The HC-SR04 Echo pin outputs 5V while the ESP32 operates at 3.3V. This was resolved by implementing a 1kΩ/2kΩ resistor voltage divider.
- pulseIn() Timeout Freezing: Without a timeout parameter, pulseIn() would hang indefinitely when no echo was received (e.g., pointing at open space > 4 metres). A 30,000 µs timeout was added to resolve this.
- PIR Sensor Physical Incompatibility: The HC-SR501 PIR sensor was initially considered but proved physically incompatible with servo mounting. The team pivoted to the HC-SR04 as the sole detection sensor.
- Alert Flooding: Telegram was receiving continuous alerts while an object remained in the threat zone. A 10-second cooldown mechanism was implemented in the Node-RED Threat Router to resolve this.

11.3 Limitations

- The HC-SR04 has a narrow beam angle (~15°). Multiple closely-spaced objects at the same distance may be detected as a single object.
- The system requires both the ESP32 and Node-RED to be on the same Wi-Fi network. External internet access requires port forwarding or a cloud MQTT broker configuration.
- The SG90 servo has a limited torque rating. Heavy or large sensor mounts could cause servo drift over time.

12. Conclusion

The Smart Ultrasonic Sensor Home Surveillance Radar System (Heimdall's Eye) successfully demonstrates how a combination of low-cost embedded hardware (ESP32, HC-SR04, SG90 servo) and IoT software platforms (MQTT, Node-RED, Telegram) can be integrated into a functional, intelligent surveillance solution.

The project achieved its full set of technical objectives: 180° radar scanning, automatic dual-mode operation, multi-level threat detection and notification, live radar dashboard visualisation, and bidirectional remote control via Telegram. All AI-generated code was independently verified through physical testing and cross-referencing with component datasheets.

The Heimdall's Eye system proves the viability of building affordable, responsive, and remotely accessible home surveillance using open IoT protocols. Future development could extend the system with camera integration, cloud logging, multiple sensor nodes for full 360° coverage, and machine learning-based object classification.

13. References

- HC-SR04 Ultrasonic Sensor Datasheet — ElecFreaks, 2023.
- ESP32 Technical Reference Manual — Espressif Systems, 2024.
- Node-RED Documentation — OpenJS Foundation, 2024. https://nodered.org/docs/
- HiveMQ Public MQTT Broker — HiveMQ, 2024. https://www.hivemq.com/
- Telegram Bot API Documentation — Telegram, 2024. https://core.telegram.org/bots/api
- Arduino IDE and ESP32 Arduino Core — Espressif Systems. https://github.com/espressif/arduino-esp32
- PubSubClient MQTT Library — Nick O'Leary, 2024. https://github.com/knolleary/pubsubclient